

Överlagring av << och >>

I våra exempelprogram hittills har vi använt medlemsvis access för att skriva ut innehållet i en `Vector`, d.v.s. vi har anropat metoderna `x()`, `y()` och `z()` för att komma åt koordinaterna i en vektor och skrivit ut dem. Vore det inte fint om klassen själv kunde skriva ut sig till skärmen eller annan stream (t.ex. en fil)? Även detta kan åstadkommas med hjälp av överlagring av operatorer. Vi måste dock även här använda en extern *funktion* som ges tillträde till `Vector` via en *friend*-deklaration. Koden kan se ut på följande sätt:

```
// friend-definition
friend ostream & operator<< (ostream & Stream, const Vector & V);

// implementation
ostream & operator<< (ostream & Stream, const Vector & V) {
    // skriv ut till den stream vi fått som parameter
    Stream << "x=" << V.x () << ", y=" << V.y () << ", z=" << V.z ();

    // returnera streamen
    return Stream;
}
```

Eftersom det är en funktion som inte tillhör klassen `Vector` måste även vektorn komma med som en parameter till funktionen. Se [Kapitel 8](#) och [Kapitel 19](#) för mera information om streams. Det kan verka konstigt att vi måste returnera `Stream` i funktionen. Det görs därför att man ju kan kombinera ihop långa utskrifter till en enda genom att placera multipla << och data emellan. Man måste returnera streamen så att nästa << skall kunna fortsätta skriva ut till en instans av `ostream`. I exemplet ovan skrivs alltså först "x=" ut, sedan returneras `Stream` till nästa variabel/konstant som skall skrivas ut, i det här fallet `V.x()`. Vad vi i praktiken använder är operatoren << som klassen `ostream` överlagrat för att möjliggöra enkel utskrift!

Vi kan nu enkelt skriva ut vektorer, t.ex. på följande sätt:

```
#include <iostream>
#include "Vector11.h"

int main () {
    Vector V1 ( 1, 2, 3 );
    Vector V2 ( 6, 5, 4 );

    // skriv ut V1 och V2
    cout << V1 << endl << V2 << endl;
}
```

Vi får då följande utskrift:

```
% ./TestVector11
x=1, y=2, z=3
x=6, y=5, z=4
%
```

Överlagrad input med >>

På motsvarande sätt som man kan överlagra << för att möjliggöra enkel utskrift av vektorer kan man överlagra >> för att möjliggöra inläsning av vektorer från tangentbord, filer eller annan stream. Här får vi dock ett problem med vår tidigare definition av operatoren << om vi vill kunna läsa in samma utskrivna data på nytt. Vi har i metoden koncentrerat oss på att göra utskriften lättläslig och informativ, men vi har samtidigt gjort den svårare att läsa in. Vi måste läsa in de i princip onödiga "x=" o.s.v. Vi är dock endast ute efter att läsa endast tre tal, så vi ändrar koden för utskriftsoperatoren en aning:

```
ostream & operator<< (ostream & Stream, const Vector & V) {
    // skriv ut till den stream vi fått som parameter
    Stream << V.x () << ' ' << V.y () << ' ' << V.z () << ' ';

    // returnera streamen
    return Stream;
}
```

Definitionen ändras inte, endast formateringen av den utskrivna texten. Vi skriver nu endast ut ett enda mellanslag som separerar de olika talen samt ett sista mellanslag efter z-koordinaten. Det sista mellanslaget hindrar att två vektorer skrivs ut fast i varandra, d.v.s. att den första vektorns z-koordinat skrivs ihop med den andras x-koordinat. Notera att vi inte behöver ändra definitionen på utskriftsmetoden ifall vi inte är inträddade av att använda enkel inläsning med >>. Klasser har oftare överlagrat << än de överlagrat >>. I dina egna klasser bör du överväga om en vacker eller lättläslig utskrift behövs. Detta gäller förstås endast då man skriver till filer, eftersom man inte kan läsa in någonting från *skärmen*.

Vi kan nu definiera operator >> som läser in data som vår modifierade utskriftsoperator ovan skrivit ut:

```
// friend-definition
friend istream & operator>> (istream & Stream, Vector & V);

// implementation
istream & operator>> (istream & Stream, Vector & V) {
    // skriv ut till den stream vi fått som parameter
    Stream >> V.m_X >> V.m_Y >> V.m_Z;

    // returnera streamen
    return Stream;
}
```

Denna överlagrade operator är implementerad enligt samma principer som överlagrade <<, men vi läser istället in i en vektor *V*. Vi märker att *V* inte är deklarerad som *const*, eftersom vi ämnar ändra dess värde. Då vi läser in behöver vi inte bry oss om att "läsa bort" de tomrum som vi skrev ut mellan vektorns koordinater, eftersom det görs automatiskt av *istream*. Vi kan således göra t.ex. följande program som skriver ut två vektorer till en fil och läser in dem igen, men i omvänd ordning (en form av *swap* utan temporär variabel...):

```
#include <stream>
#include "Vector12.h"

int main () {
    Vector V1 ( 1, 2, 3 );
    Vector V2 ( 6, 5, 4 );

    // öppna en fil
    ofstream Ut ( "VektorData.txt" );
```

```
// skriv ut V1 och V2
Ut << V1 << endl << V2;

// stäng utfilen
Ut.close ();

// öppna filen på nytt för läsning
ifstream In ( "VektorData.txt" );

// läs in till vektorerna, men i omvänd ordning
In >> V2 >> V1;

// skriv ut på skärmen
cout << "V1: " << V1 << endl << "V2: " << V2 << endl;
}
```

Kör vi programmet får vi följande utskrift på skärmen:

```
% ./TestVector12
V1: 6 5 4
V2: 1 2 3
%
```

Vi märker att `V1` och `V2` bytt värden, eftersom vi läste in dem i omvänd ordning mot när vi skriv ut dem.

[Förutgående](#)

Operatorerna == och =

[Hem](#)
[Upp](#)

[Nästa](#)
Överlagring av
typkonverteringar